

HASKELL - LESSON 1

Setup, mental model, types, functions and decisions

QUICK REFERENCE | 1/2

1. START IN WSL

SETUP

Check installation

```
ghci
```

If GHCi is missing on Ubuntu WSL:

```
sudo apt update
sudo apt install ghc
```

Create a workspace

```
mkdir haskell-practice
cd haskell-practice
code .
```

Create **Lesson1.hs** in VS Code.

2. RUN AND RELOAD

WORKFLOW

From the folder containing the file:

```
ghci Lesson1.hs
```

After editing: save with **Ctrl+S**, then enter:

```
:r
```

Test a loaded function:

```
double 6 -- 12
```

GHCI COMMANDS

REFERENCE

:l File.hs - load a file
:r - reload saved changes
:t item - show its type
:q - quit GHCi

THE HASKELL MINDSET

CONCEPT

- A function turns inputs into an output.
- Expressions evaluate to values.
- Every value has a type.
- Values do not normally change.
- Lazy evaluation computes only what is needed.

```
double x = x * 2
```

```
double 5
= 5 * 2
= 10
```

CORE TYPES

TYPES

Int - fixed-range whole number

Integer - arbitrary-size whole number

Float / Double - decimal number

Char - one character, e.g. 'A'

String - text, e.g. "Hello"

Bool - True or False

[Int] - list of integers

```
:t "Hello"
-- "Hello" :: String
```

OPERATORS

SYNTAX

Math: + - * / ^ `div` `mod`

Compare: == /= < <= > >=

Logic: && || not

```
17 `div` 5 -- 3
17 `mod` 5 -- 2
5 > 2 && 8 /= 3 -- True
```

FUNCTIONS + SIGNATURES

CODE

The final type is the return type.

```
double :: Int -> Int
double x = x * 2
rectangleArea :: Float -> Float -> Float
rectangleArea width height = width * height
```

Call functions with spaces:

```
rectangleArea 4.0 5.0 -- 20.0
```

No comma-separated call syntax is needed.

IF / THEN / ELSE

DECISION

```
numberSize :: Int -> String
numberSize x =
  if x > 100
  then "Large"
  else "Small"
```

Rules: else is required, and both branches must return compatible types.

A comparison already returns **Bool**:

```
isAdult age = age >= 18
```

NAMING + LAYOUT

REMEMBER

- Functions and values start lowercase.
- Types start uppercase: Int, Bool, String.
- Haskell is case-sensitive.
- Indent related lines consistently.
- Single-line comments start with --.

HASKELL - LESSON 1

Lists, strings, ranges, common functions and debugging

QUICK REFERENCE | 2/2

LIST BASICS

LISTS

A list is ordered and all its elements have the **same type**.

```
numbers :: [Int]
numbers = [1,3,5,7]
```

```
empty = []
```

Cons (:) adds one element to the front:

```
0 : [5,10,15]
-- [0,5,10,15]
```

Concatenate (++) joins two lists:

```
[1,2] ++ [3,4]
-- [1,2,3,4]
```

THE : VS ++ RULE

KEY RULE

one element : list

list ++ list

```
1 : [2,3] -- valid
[1] ++ [2,3] -- valid
0 ++ [5,10,15] -- invalid
```

[1,2,3] is shorthand for 1 : 2 : 3 : [].

INDEXING

!!

Indexes start at 0.

```
[10,20,30] !! 1 -- 20
```

Nested list example:

```
scores = [[50,60],[80,90]]
(scores !! 1) !! 1 -- 90
```

An index outside the list causes a runtime error.

ESSENTIAL LIST FUNCTIONS

REFERENCE

```
xs = [2,4,6,8]
```

```
head xs = 2 | tail xs = [4,6,8]
```

```
last xs = 8 | init xs = [2,4,6]
```

```
length xs = 4 | null xs = False
```

```
reverse xs = [8,6,4,2]
```

```
take 2 xs = [2,4]
```

```
drop 2 xs = [6,8]
```

```
maximum xs = 8 | minimum xs = 2
```

```
sum xs = 20 | product xs = 384
```

```
elem 4 xs = True
```

```
4 `elem` xs -- True
```

STRINGS ARE [CHAR]

TEXT

```
type String = [Char]
```

List functions therefore work on text:

```
head "Haskell" -- 'H'
```

```
tail "Haskell" -- "askell"
```

```
reverse "drawer" -- "reward"
```

```
'H' : "ello" -- "Hello"
```

RANGES + LAZINESS

RANGES

```
[1..5] -- [1,2,3,4,5]
```

```
[2,4..10] -- [2,4,6,8,10]
```

```
['a'..'e'] -- "abcde"
```

Infinite range - request only what you need:

```
take 5 [1..]
```

```
-- [1,2,3,4,5]
```

COMMON ERRORS

DEBUG

Mixed list types

```
[1,2,"3"] -- invalid
```

Different if-branch types

```
if True then 10 else "ten" -- invalid
```

Wrong operator shape

```
0 ++ [5,10] -- 0 is not a list
```

Empty-list failures

```
head []
```

```
last []
```

Changes not appearing? Save the file, then enter :r.

MINI EXAMPLE

PUT IT TOGETHER

```
studentName :: String
studentName = "Iqra"
```

```
marks :: [Int]
marks = [68,75,81,59,72]
```

```
resultMessage :: String
resultMessage =
  if head marks >= 50
  then studentName ++ " passed"
  else studentName ++ " failed"
```

Useful checks in GHCi:

```
sum marks -- 355
maximum marks -- 81
resultMessage -- "Iqra passed"
```

READY FOR LESSON 2?

SELF-CHECK

- Load, reload and test a .hs file.
- Read a simple type signature.
- Write a function and an if expression.
- Explain : versus ++.
- Use list functions and finite ranges.

Next: tuples and list comprehensions.